

4. System Analysis & Design Restaurant Finder Web Application

4.1 Problem Statement & Objectives

Problem Statement

Finding suitable restaurants can be a challenging and time-consuming task due to the large number of available options and the fragmentation of information across multiple platforms. Users often struggle to compare restaurants based on location, ratings, price range, and services. Additionally, the lack of a centralized and user-friendly system makes it difficult for users to make quick and informed decisions. Therefore, there is a need for a web-based application that simplifies discovering, filtering, and evaluating restaurants efficiently.

Objectives

- Develop a user-friendly web application for restaurant search
- Provide advanced filtering based on category, rating, price, and location
- Display detailed restaurant information including ratings and menus
- Integrate external APIs (e.g., Yelp API) for real-time data
- Ensure responsive design across all devices
- Build a scalable and maintainable frontend using React
- Enhance user experience through intuitive UI design

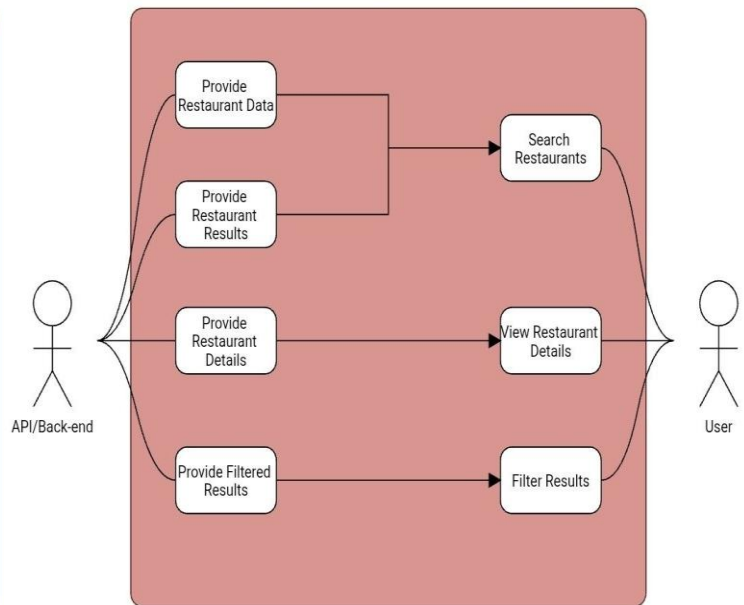
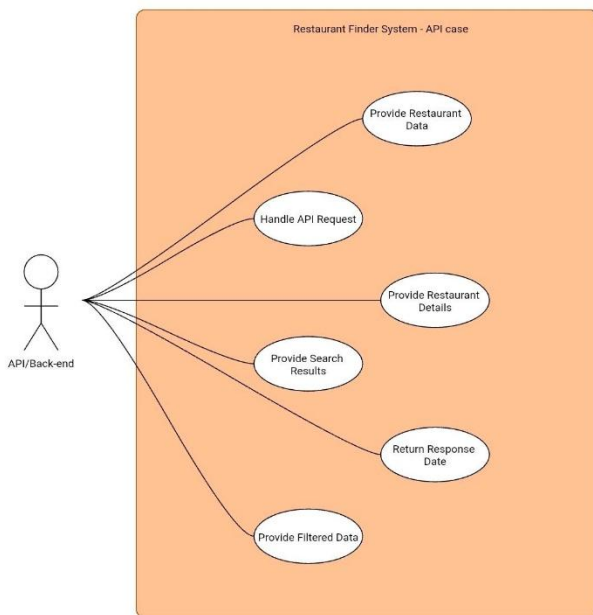
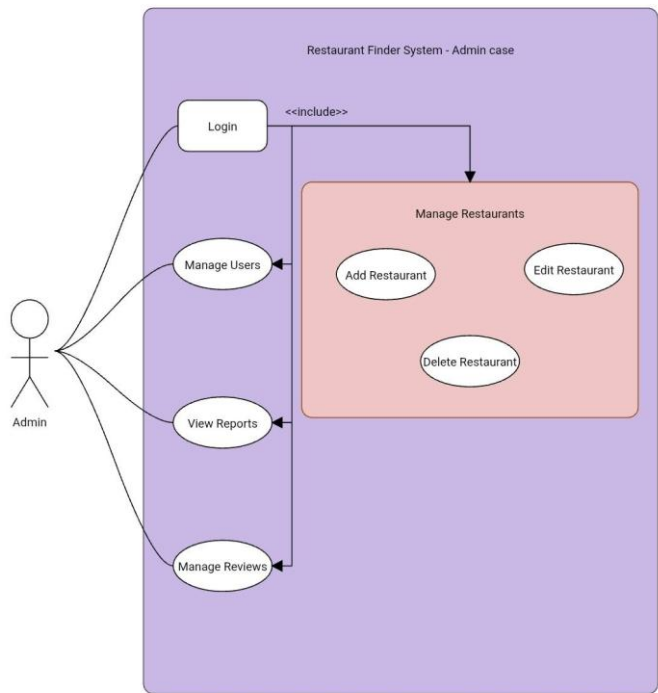
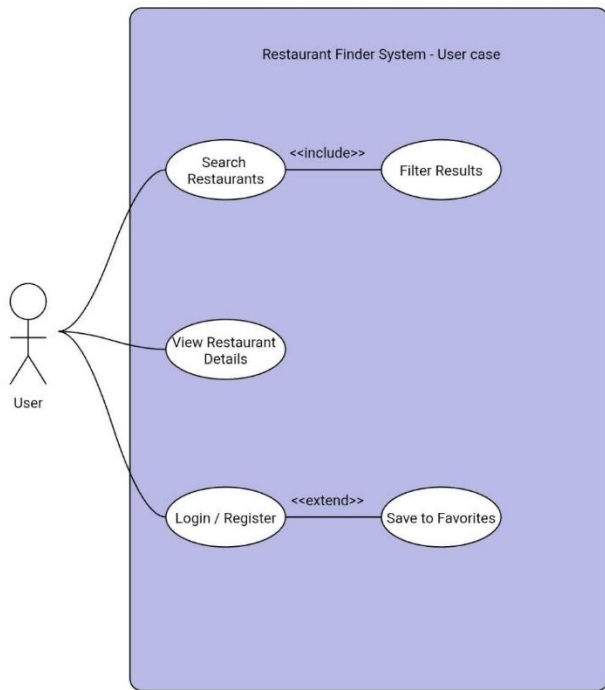
4.2 Use Case Diagram & Descriptions

Actors

- User: Interacts with the system to search and explore restaurants
- Admin: Manages system data and monitors performance
- External API (Yelp API): Provides restaurant data

Use Cases

- Search Restaurants
- Fetch Restaurant Data (API)
- View Restaurant Details
- Filter Restaurants
- Login / Register
- Save Favorites



Use Case Descriptions

- Search Restaurants: Users search by name, category, or location.
- Fetch Restaurant Data: System retrieves data from Yelp API.
- View Details: Displays ratings, menu, and reviews.
- Filter Restaurants: Users refine results by preferences.
- Login/Register: Users create accounts and log in securely.
- Save Favorites: Users save restaurants for later.

4.3 Functional & Non-Functional Requirement

Functional Requirements

- Search restaurants by name, category, or location
- Display restaurant details (ratings, menu, hours)
- Filter search results
- User authentication (login/register)
- Save favorite restaurants

ID	Feature	Description
The system shall allow users to search for restaurants by name, cuisine type, or location.	Search Functionality	FR-01
The system shall display detailed information including menus, ratings, operating hours, and photos.	Restaurant Profiles	FR-02
Users shall be able to filter search results by category, price range, and minimum rating.	Advanced Filtering	FR-03

Non-Functional Requirements

- Performance: Load within 2–3 seconds
- Usability: Simple and intuitive interface
- Responsiveness: Works on all devices
- Security: Protect user data (HTTPS)
- Reliability: High availability (99%)

Specification	Requirement	Category
The system should load pages and search results within less than 3 seconds.	Response Time	Performance
The user interface must be fully compatible with mobile, tablet, and desktop screens.	Responsive Design	Usability
User credentials and personal data must be encrypted using industry-standard protocols (e.g., SSL/HTTPS).	Data Protection	Security
The system should be operational 99.9% of the time to ensure constant access for users.	High Availability	Reliability

4.4 Software Architecture

Overview

The system follows a client-server architecture separating frontend, backend, and external services. This enhances scalability, maintainability, and performance. The frontend is built with React, while the backend exposes RESTful APIs and integrates with Yelp API.

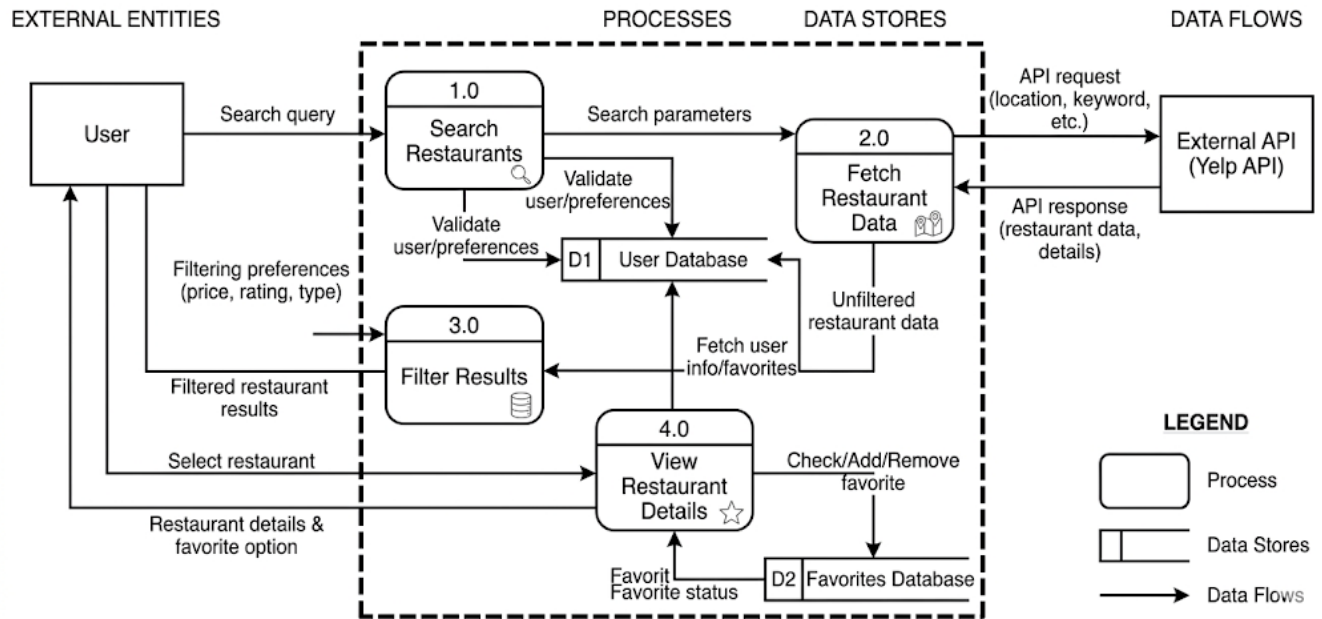
System Components

- Frontend (React): UI rendering and user interaction
- Backend (API): Handles logic and data processing
- Database: Stores user and system data
- External API: Provides real-time restaurant data

System Workflow

- User interacts with frontend
- Frontend sends request to backend
- Backend processes request
- Backend may call external API
- Data returned to frontend
- Frontend displays results

Level 1 Data Flow Diagram: Restaurant Finder App



Architecture Style

- Component-Based Architecture (React)
- Client-Server Architecture
- MVC-inspired design
- RESTful API communication

